

Report

Main Report: **REPORT.md**

1. **Implementation Overview** (Section 1)

- Complete ReactAgent architecture
- Response parser implementation
- LLM integration details
- Environment and tools

2. **Performance Results** (Section 2)

- **Accuracy: 25% (5/20 instances resolved)**
- Completion rate: 85% (17/20 completed)
- Detailed breakdown of resolved vs unresolved instances
- Analysis of successful resolutions

3. **Custom Tools Development** (Section 3)

- **5 custom tools** implemented beyond requirements:
 - `show_file` - Display code with line numbers
 - `replace_in_file` - Precise line-based editing
 - `find_in_file` - Search within files
 - `write_file` - Create new files
 - `generate_patch` - Git patch generation
- Rationale for each tool design
- Implementation highlights
- Tool usage patterns

4. **Lessons Learned** (Section 5)

- Technical insights (prompt engineering, tool design)
- LLM behavior observations (strengths & weaknesses)
- Architecture lessons
- Evaluation process insights

5. **Future Improvements** (Section 6)

- Short-term quick wins
- Medium-term research directions

- Long-term open challenges

6. Appendices

- Appendix A: Detailed analysis of all 5 resolved instances
- Appendix B: Tool usage statistics



Key Metrics

Metric	Value
Accuracy	25% (5/20 resolved)
Completion Rate	85% (17/20 completed)
Error Rate	15% (3/20 errors)
Custom Tools Created	5
Total Tool Calls	~900 across all instances



Resolved Instances

1. `django__django-10973` - PostgreSQL authentication fix
2. `django__django-11179` - Fast delete primary key clearing
3. `django__django-13810` - Middleware exception handling
4. `django__django-14053` - Static files post-processing
5. `sympy__sympy-24213` - Unit system dimension equivalence



Evaluation Results

The official evaluation results are in: `gpt-5-mini.my_evaluation_run.json`

This file contains:

- Total, submitted, completed, resolved instance counts
- Complete lists of resolved/unresolved/error instance IDs
- Schema version for compatibility



Assignment Requirements Met



Core Architecture

- ReactAgent class with all required attributes and methods
- Message tree structure with backtracking support
- Tool registration and auto-documentation

✓ Response Parser

- Custom textual format (non-JSON/XML)
- Handles multiline arguments
- Robust delimiter parsing with rfind

✓ LLM Implementation

- Abstract LLM base class
- OpenAIModel subclass for GPT-5-mini
- Integration with medium reasoning mode

✓ Tools

- Required: `run_bash_cmd`, `finish`, `add_instructions_and_backtrack`
- Enhanced: 5 additional custom tools for better accuracy

✓ Evaluation Setup

- GPT-5-mini (medium reasoning) configured
- Baseline and enhanced evaluations completed
- Results measured and reported



Ready for Submission

The following files are ready for submission to the course server:

1. Code Artifact (ZIP) - Entire repository

- All source code in place
- `requirements.txt` for dependencies
- `README.md` with setup instructions
- No secrets/keys committed

2. Report (PDF) - Convert `REPORT.md` to PDF

- Contains all required sections
- Accuracy numbers reported
- Custom tools described with rationale
- Lessons learned documented

3. Evaluation Results (JSON) - `gpt-5-mini.my_evaluation_run.json`

- Official SWE-Bench format
- Complete metrics
- Instance-level details

Key Takeaways

What Worked Well

- Well-designed tools matching LLM capabilities
- Robust error handling preventing crashes
- Tree-based message history for clean backtracking
- Systematic exploration patterns

Main Limitations

- Model reasoning depth for complex multi-file bugs
- Context window constraints
- Lack of iterative refinement (one-shot attempts)
- Backtracking underutilized by the model

Recommendations

- Explore multi-agent architectures (Explorer, Analyzer, Patcher, Verifier)
- Implement iterative debugging with test feedback loops
- Add semantic code search capabilities
- Fine-tune on successful solution trajectories



Comparison Context

For reference, typical performance on SWE-Bench Lite subset:

- Random baseline: ~0%
- SWE-agent (best): ~12-18%
- no backtrack implementation: **25%** ✨
- backtrack implementation: **25%** ✨

We find that backtracking is severely underutilized by the model.